The Planner Step by Step: What Goes In and What Comes Out

November 2026

This note follows `planner.run(seed, p)` in `Dubins_TSP_New_Implementation_claude/`, the program that carries out the overall algorithm of Section 0.11 of `Dubins_TSP.pdf`. For every step it states what goes in, what the step does, and what comes out, with the name and size of each piece of data as the code passes it on. The steps are numbered 0 to 6, as in the flow diagram `output/pipeline_flow.png`; each step also gives the steps of Section 0.11 it carries out. The example numbers come from seed 1 with the default parameters (Table 3).

Notation. The base station is S_0 and the sensors are $S_1, \ldots, S_n$. In the code, sensor i is counted from 0, so the paper's S_{i+1} is the code's sensor i; in the distance table D, row and column 0 belong to the base and row and column $i+1$ to sensor i. A group of sensors is a set $\mathcal{S}_k$, and a tour π is the order in which one sink visits its group, starting and ending at S_0. B is the time budget; in Step 3c the two halves of a split group are written A_t and B_t, as in Section 0.9.

1 The Steps at a Glance

Step	Code	Sec. 0.11	Input	Output
0	`scenario.make`	–	seed; map settings	map: base, n sensors, dwell times τ, obstacles
1	`geodesic.build`	Step 2	base, sensors, obstacles; margins r_{plan}, $r_{\text{safe}} + 0.5$	distance table D ($(n{+}1) \times (n{+}1)$), predecessors, graph nodes
2	`planner.plan_routes`	Step 1	all sensor ids	a queue holding one group of all n sensors
3a	`ga_tour.optimize`	Steps 3, 6	one group $\mathcal{S}_k$; positions; r; R	visiting order π
3b	`trajectory.build`	Steps 2, 4, 6	order π; D and paths; τ; R, v, Δt	tour: path, frames, length L_k, time T_k
3c	`partition.bisect`	Steps 5, 7	T_k, B; for a split: $\mathcal{S}_k$, D, τ	keep the tour as a sink, or two halves A_t, B_t to the queue
4	`conflict_game.resolve`	Steps 8–10	frames, T_k, L_k of all K sinks; d_{safe}, B	new frames, T_k, L_k; one record per game
5	`planner.reoptimize`	Step 12	sinks still in conflict; their tours	new tours; game played again (at most 3 rounds)
6	`planner.check`	Step 11	the final result	K, $T_{\max}$, L_{total} and every constraint measured

Table 1: Every step with its code, its steps in Section 0.11, and what goes in and out. Steps 2–3c repeat until the queue is empty; Steps 4–5 repeat until no conflict is left or three repairs have been tried.

2 The Data Passed Between Steps

Name in the code	Made by	What it holds	Used by
`scen`	Step 0	`base` $\in \mathbb{R}^2$; `xy` $\in \mathbb{R}^{n\times 2}$, the sensor positions; `tau` $\in \mathbb{R}^n$, the dwell times τ_i; `obstacles`, a list of polygons (each an $m \times 2$ array of corners)	1, 3, 6
`geo`	Step 1	`D` $\in \mathbb{R}^{(n+1)\times(n+1)}$, the geodesic distances d_{geo}; `pred`, the predecessors of Dijkstra's runs; `nodes` $\in \mathbb{R}^{N\times 2}$, the graph's points; `obstacles`, the union of the original obstacles	3, 4, 6
r	from D	$r_i = D[0, i{+}1]$, each sensor's distance from the base (eq. 9)	3a, 3c
D_s	from D	$D[1{:}, 1{:}]$, the sensor-to-sensor distances	3c
`ids`	Step 2 or 3c	a group $\mathcal{S}_k$: the ids of its sensors	3a, 3c
`order`	Step 3a	the tour π: the group's ids in visiting order	3b
`tour`	Step 3b	`order`; `path`, a dense polyline (points $\Delta s = 0.25$ m apart); `frames` $\in \mathbb{R}^{F\times 2}$, the position every $\Delta t = 0.5$ s from $t = 0$; `L`, the length L_k [m]; `T`, the mission time T_k [s]	3c, 4, 5, 6
`tours`	Steps 2–3c	the K tours kept, one per sink	4, 5, 6
`result`	Steps 4–5	`tours` (the plan before resolving); `frames`, `T`, `L` after resolving; `games`; the conflicts `before` and `after`; the repair `round` kept	6
`checks`	Step 6	K, $T_{\max}$, L_{total}, and every constraint measured	the user

Table 2: The data that flows through the planner, with its size. n is the number of sensors, N the number of graph points, K the number of sinks and F the number of frames of a sink.

`planner.run` hands all of it back as one dictionary: `scenario`, `geo`, `result`, `checks`, `times` and `params`.

3 Each Step in Detail

Step 0. Make the map (`scenario.make`)

Input.

- The seed, a whole number. The same seed always gives the same map.
- From `config.py`: a 100×100 m square with the base at $(50, 50)$; 10–15 obstacles; 90–110 sensors; dwell times τ_i drawn from 1–5 s; 60 % of the sensors placed 3–7 m from an obstacle, with half of those placement attempts aimed at inner corners.

What it does.

1. Places random obstacles (polygons, L, C and cross shapes), each centred at least 25 m from the base, so that their r_{plan}-inflated versions neither touch each other nor come within 5 m of the base.
2. Places the sensors near obstacles, and keeps one only if a sink can drive base $\to$ sensor $\to$ base keeping r_{safe} from every obstacle (this uses Steps 1 and 3b on a small test map).
3. Spreads the remaining sensors over the free space and draws a dwell time for each.

Output. `scen`: `base`, `xy` $(n \times 2)$, `tau` (n), `obstacles`, and `n_near`, the number of sensors placed near obstacles. Step 0 is not part of Section 0.11: in the field the map is given.

On seed 1. 9 obstacles and $n = 98$ sensors.

Step 1. Geodesic distances (`geodesic.build`; Section 0.11, Step 2)

Input.
- `base`, `xy` and `obstacles` from Step 0.
- The margins $r_{\text{plan}} = r_{\text{safe}} + R = 2 + 5 = 7$ m and $r_{\text{safe}} + 0.5 = 2.5$ m.

What it does.
1. Inflates every obstacle by r_{plan}; the corners of the inflated polygons become graph points, after the $n+1$ terminals $(S_0, \ldots, S_n)$.
2. Joins two points by an edge if the straight line between them does not enter an obstacle inflated by $r_{\text{safe}} + 0.5$ (when the line touches a terminal) or by r_{plan} (between two corners). The edge's weight is its length. This is the visibility graph (`geodesic.visibility_graph`).
3. Runs Dijkstra's algorithm from each of the $n+1$ terminals. Each run gives one row of D and the predecessors from which `geodesic.path` rebuilds the corner-to-corner path. The graph itself is not kept.

Output. `geo`: `D` ($(n{+}1) \times (n{+}1)$, symmetric, never below the straight-line distance), `pred`, `nodes`, `n_terminals` $= n+1$, and `obstacles`. From D the later steps take $r = D[0, 1{:}]$ and $D_s = D[1{:}, 1{:}]$. If a sensor cannot be reached, the planner stops with an error.

On seed 1. 99 terminals, 148 corners, 7898 edges. The longest detour is to S_{54} (the code's sensor 53): $r = 55.0$ m against 41.8 m in a straight line.

Step 2. The queue of groups (`planner.plan_routes`; Section 0.11, Step 1)

Input. The ids $0, \ldots, n-1$ of all sensors.

What it does. Starts a first-in, first-out queue holding one group, $\mathcal{S}_1 = \{0, \ldots, n-1\}$: one sink with every sensor. Steps 3a–3c then take groups from the front of the queue until it is empty.

Output. The queue $[\mathcal{S}_1]$, an empty list of kept tours, and an empty list of problems.

Step 3a. Visiting order: the genetic algorithm (`ga_tour.optimize`; Section 0.11, Steps 3 and 6)

Input.
- The group $\mathcal{S}_k$ taken from the front of the queue.
- The sensor positions `xy`, the base, and r from Step 1.
- $R = 5$ m; a population of 50 tours; 100 generations; mutation probability 0.1; the random-number generator of Steps 1–7, started from the seed.

What it does.
1. Makes 50 start tours that are radially unimodal (Section 0.8.2): each sensor is put at random in S_{out} or S_{in}; S_{out} is visited by increasing r, then S_{in} by decreasing r, so every start tour has one peak.
2. Scores each tour by its fitness (Section 0.8.1)

$$\text{Fitness}(\pi) = L_{\text{euclid}}(\pi) + m(\pi)\,\pi R,$$

   the straight-line length plus $\pi R \approx 15.7$ m for each radial peak $m(\pi)$. Obstacles enter only through r.
3. In each generation keeps the better 25 tours and adds 25 children: ordered crossover of two kept tours (the first c sensors of one, then the rest in the other's order), and with probability 0.1 a swap of two sensors. The paper does not specify these two moves; they are the code's choice.
4. After 100 generations returns the tour with the lowest fitness. A group of one or two sensors needs no search: it is visited in order of r.

Output. The visiting order π of $\mathcal{S}_k$ (a permutation of its ids). Extra peaks cost πR each but are allowed, so π need not be unimodal.

On seed 1. The first run (all 98 sensors) ends with a best tour of 9 peaks.

Step 3b. The drivable path and its time (`trajectory.build`, with `dubins_path`; Section 0.11, Steps 2, 4 and 6)

Input.

- The order π from Step 3a.
- From Step 1: `geo` (the corner-to-corner paths through `geodesic.path`, and the obstacles).
- From Step 0: the dwell times τ_i.
- $R = 5$ m, $v = 5$ m/s, $\Delta t = 0.5$ s, $\Delta s = 0.25$ m, $r_{\text{safe}} = 2$ m, and the Dubins method (`shortest` by default, or `pdf` for the rule of Section 0.13).

What it does.

1. *Waypoints:* base, the sensors in order, base, joined by their geodesic paths; the corners of those paths become waypoints too. A corner closer than R to a neighbouring waypoint is dropped if the shortcut stays $r_{\text{safe}} + 0.5$ m from every obstacle.
2. *Headings* (the θ of Section 0.5): at every waypoint one of 17 candidates (16 directions and the bisector of the turn) is chosen by dynamic programming, so that the total Dubins length is as short as possible and every leg keeps r_{safe} from the obstacles.
3. *Legs:* every pair of consecutive poses is joined by a Dubins path (`dubins_path.returnDubinsPath`): the shortest of LSL, RSR, LSR, RSL, RLR and LRL, or the type of the Section 0.13 rule.
4. *Time* (eq. 17): driving at constant speed and stopping τ_i at each sensor,

$$T_k = \frac{L_k}{v} + \sum_{i \in \mathcal{S}_k} \tau_i .$$

5. *Frames:* the position is read every Δt seconds from $t = 0$; during a stop the position repeats.

Output. `tour`: `order`, `path`, `frames` ($F \times 2$), `L` $= L_k$ and `T` $= T_k$.

On seed 1. Sink 8's 5 sensors: 120.5 m of straight lines (what the GA scored), a Dubins path of 176.9 m, and $T = 176.9/5 + 15.6 = 51.0$ s over 103 frames.

Step 3c. The budget, and splitting (`partition.bisect`; Section 0.11, Steps 5 and 7)

Input.

- T_k from Step 3b and the budget $B = 80$ s (eq. 18); $K_{\max} = 30$.
- For a split: the group $\mathcal{S}_k$, `xy`, the base, D_s and r from Step 1, τ, v and R.

What it does.

- If $T_k \le B$, the tour is kept: one more sink.
- If $T_k > B$ and $\mathcal{S}_k$ has more than one sensor, the group is split in two (Section 0.9.1):
  1. affinity $W_{ij} = \exp\big(-(d_{\text{geo}}(i,j)/\sigma)^2\big)$, $W_{ii} = 0$, with $\sigma = \max(\text{mean nearest-neighbour distance}, R)$;
  2. degrees $d_i = \sum_j W_{ij}$ and the normalised Laplacian $L_{\text{norm}} = I - D^{-1/2} W D^{-1/2}$;
  3. its Fiedler vector f, the eigenvector of the second-smallest eigenvalue;
  4. for every threshold t, $A_t = \{S_i : f_i \le t\}$ and $B_t = \{S_i : f_i > t\}$; the threshold kept minimises $\max\big(\hat{T}(A_t), \hat{T}(B_t)\big)$, where $\hat{T}$ is the length of a sweep round the base in polar-angle order with geodesic legs, divided by v, plus $\sum \tau_i$.

Both halves go to the back of the queue; each later gets its own Steps 3a and 3b. The other groups are not touched.
- If a single sensor alone needs $T_k > B$, its tour is kept and the problem is reported.
- If the kept tours plus the waiting groups would need more than $K_{\max}$ sinks, planning stops and the problem is reported.

Output. Either the tour, added to the kept tours, or the two halves A_t and B_t, added to the queue. When the queue is empty, Steps 1–7 hand on `tours` (K tours, each with $T_k \le B$) and the list of problems.

On seed 1. One sink with all 98 sensors would need 1202 s. After 29 GA runs and 14 splits there are $K = 15$ sinks, the longest taking $T_{\max} = 79.6$ s. The first split gives halves of 52 and 46 sensors.

Step 4. The conflict game (`conflict_game.resolve`; Section 0.11, Steps 8–10)

Input.
- The `frames`, T_k and L_k of all K kept tours (all sinks leave the base at $t = 0$).
- `geo`, for the obstacle check of a DETOUR.
- $d_{\text{safe}} = 3$ m (eq. 20); the 5 m docking zone round the base; the cost weights $(\beta, \gamma, \delta) = (1, 1, 1)$; a longest WAIT of 20 s; a DETOUR side-step of $d_{\text{safe}} + 0.5 = 3.5$ m; the budget B; at most 500 games.

What it does.
1. Finds the earliest moment when two sinks come closer than d_{safe} (measured between frames, outside the docking zone). The sinks in conflict then are the players.
2. Gives each player its strategies: GO (keep the plan), WAIT (the shortest stop that clears the conflict, or else the least risky one) and DETOUR (step aside and rejoin). A WAIT or DETOUR is offered only if the sink still finishes within B.
3. Each player's cost is

$$J_k = \alpha \cdot (\text{seconds closer than } d_{\text{safe}} \text{ to any sink}) + \beta\,\Delta T + \gamma\,\Delta L/v + \delta\,W,$$

   with α large enough that any danger outweighs any delay. Starting from all GO, the player who gains most switches, until no player can gain: a pure Nash equilibrium (Section 0.10.1).
4. Applies the chosen strategies and moves on to the next conflict.

Output. New `frames`, T_k and L_k (a WAIT adds time; a DETOUR adds time and length), and one record per game: its players, their strategies, the switches, the potential Φ after each switch, and the equilibrium. With `--resolver simple`, `simple_resolver.resolve` (Section 0.10.2) takes the same input and gives the same output, with one record per stop.

On seed 1. 6 games take the plan from 16 pairs and 63.0 s in conflict to none.

Step 5. Repair (`planner.reoptimize`, inside `planner.resolve_and_check`; Section 0.11, Step 12)

Input.
- The pairs still in conflict after Step 4.
- The tours of the sinks involved, the map, `geo`, B, and a second random-number generator, started from the seed plus 1000, so that both resolvers start Steps 8–12 alike.

What it does.
1. For each sink still in conflict, tries two other tours: its order reversed, and a fresh GA run on its sensors. Each is built as in Step 3b, and kept only if it fits B and is less dangerous against the other sinks.
2. Plays Step 4 again on all the planned tours, with the new ones.
3. Repeats at most 3 times, stopping as soon as no conflict is left.

Output. `result`, from the round with the least time in conflict: `round`, `tours` (the plan before resolving), `frames`, `T` and `L` after resolving, `games`, and the conflicts `before` and `after`. Conflicts are reported, not guaranteed to be removed: on seeds 1–30 the game leaves some on 8 maps.

On seed 1. No repair is needed; round 0 is kept.

Step 6. The checks (`planner.check`; Section 0.11, Step 11)

Input. `result` from Step 5, `scen`, `geo`, the parameters, and the problems found in Step 3c.

What it does. Measures every objective and constraint of Sections 0.6–0.7 on the final frames, independently of how they were made.

Output. `checks`:

- the objectives: K, $T_{\max} = \max_k T_k$, $L_{\text{total}} = \sum_k L_k$ (eqs. 11–14);
- `coverage_ok`: every sensor visited exactly once (eqs. 15–16);
- `budget_ok`: every $T_k \le B$ (eq. 18);
- `clearance`: the closest any sink comes to an obstacle, which must be at least r_{safe} (eq. 19);
- `curvature_x_R`: the sharpest bend times R, which must be at most 1.15;
- `conflict_pairs` and `closest`: the pairs still closer than d_{safe}, and the closest approach (eq. 20);
- `K_ok` ($K \le K_{\max}$) and the list of problems.

On seed 1. $K = 15$, $T_{\max} = 79.6$ s, $L_{\text{total}} = 2927$ m, no conflict left, closest approach 3.16 m.

4 How the Steps Connect

- **Run once:** Steps 0 and 1 at the start, Step 6 at the end.
- **The splitting loop** (Steps 2–3c): take a group from the front of the queue, order it (3a), build its path (3b), then keep it or split it (3c). A split puts both halves at the back of the queue. The loop ends when the queue is empty, and its output is the K tours.
- **The repair loop** (Steps 4–5): play the game on all K tours (4); if conflicts are left, re-plan the sinks involved (5) and play again. At most 3 repairs; the round with the least time in conflict is kept.
- **Three measures of length:** the GA scores straight lines plus a peak penalty (only r is geodesic); $\hat{T}$ in Step 3c uses geodesic legs; only the budget test uses the real Dubins length from Step 3b.
- **Randomness:** one random-number generator, started from the seed, serves Steps 1–7 (Steps 2–3c here); a second, started from the seed plus 1000, serves Steps 8–12 (Steps 4–5). The same seed always gives the same result.

Appendix: the Parameters Used

Symbol	Code	Value	Meaning
v	`v`	5 m/s	constant speed of a sink
R	`R`	5 m	minimum turning radius, wheelbase/ $\tan \delta_{\max} = 2.5/0.5$ (eq. 7)
r_{safe}	`r_safe`	2 m	obstacle clearance, half the width plus 1 m (eq. 5)
r_{plan}	`r_plan`	7 m	obstacle inflation for planning, $r_{\text{safe}} + R$
d_{safe}	`d_safe`	3 m	minimum distance between two sinks (eq. 20)
B	`B`	80 s	time budget of every sink (eq. 18)
$K_{\max}$	`K_max`	30	most sinks allowed
–	`base_radius`	5 m	docking zone round the base
Δt	`dt`	0.5 s	time between frames
Δs	`ds`	0.25 m	spacing of the points of a Dubins path
–	`pop_size`, `generations`, `mutation`	50, 100, 0.1	the GA
(β, γ, δ)	`weights`	$(1, 1, 1)$	weights of extra time, extra distance and waiting
–	`max_wait`, `detour_margin`	20 s, 0.5 m	the longest WAIT; DETOUR side-step $d_{\text{safe}} + 0.5$
–	`curvature_tol`	0.15	allowance when curvature is measured from sampled points
–	`max_games`, `repair_rounds`	500, 3	limits of Steps 4 and 5

Table 3: The default parameters, from `config.default_params`.